Comunicação de emergência sem infraestrutura: transmissão periódica de localização via LoRa para apoio a operações de busca e resgate

Felipe Ros Segundo Simão[1], **Robson Costa**[1]

[1]Instituto Federal de Santa Catarina (IFSC)
R. Heitor Villa Lobos, 225, São Francisco CEP: 88506-400 – Lages – SC – Brazil

felipe.rss@aluno.ifsc.edu.br,

robson.costa@ifsc.edu.br

1. Introdução

Desastres naturais como enchentes, deslizamentos de terra, ciclones e incêndios florestais representam uma ameaça recorrente em diversas regiões do mundo, causando perdas humanas, destruição de patrimônio e colapso de serviços essenciais. No Brasil, esse cenário é particularmente relevante: nos últimos anos, estados como Santa Catarina e Rio Grande do Sul têm enfrentado eventos climáticos extremos com frequência crescente, incluindo enchentes severas, ciclones extratropicais e deslizamentos que afetaram milhares de pessoas [Brasil, 2024b]. Em 2024, o país registrou ainda cerca de 30,8 milhões de hectares queimados, área 62% superior à média histórica desde 1985, com recorde de área queimada na Mata Atlântica [MapBiomas, 2025]. Esses eventos atingem não apenas cidades e lavouras, mas também pessoas que se encontram em áreas naturais e isoladas, como praticantes de trilhas, moradores de zonas rurais, trabalhadores do campo e brigadistas.

Uma das consequências mais críticas desses eventos é a interrupção das redes de telecomunicações. Estudos demonstram que a destruição de torres de celular, cabos de fibra óptica e estações-base, somada à queda no fornecimento de energia elétrica, pode comprometer gravemente a capacidade de comunicação nas áreas atingidas [El Khaled and Mcheick, 2019, Goos et al., 2024]. Durante o furacão Katrina, por exemplo, mais de mil torres de celular foram destruídas, deixando centenas de milhares de pessoas sem serviço telefônico por semanas [Townsend and Moss, 2005]. Essa interrupção prejudica diretamente a comunicação entre a população afetada e os órgãos de resposta [Chang et al., 2007].

Em áreas remotas, a comunicação é precária mesmo em condições normais. Pelo critério da Anatel, um município é considerado coberto pela telefonia móvel quando ao menos 80% da área urbana de seu distrito sede recebe sinal [Agência Nacional de Telecomunicações (Anatel), 2021], o que deixa de fora grande parte de trilhas, parques, serras e zonas rurais. O Plano Estrutural de Redes de Telecomunicações de 2026 aponta cerca de 26,4 mil localidades sem cobertura 4G e sem qualquer obrigação regulatória de atendimento [Agência Nacional de Telecomunicações (Anatel), 2026b]. Nessas situações, o fator mais crítico para as equipes de resgate é saber onde está a vítima e qual é a sua condição. Em maio de 2026, um jovem que se perdeu em uma trilha no Morro Pelado, entre Joinville e Campo Alegre (SC), permaneceu desaparecido por cinco dias; a busca mobilizou mais de 60 pessoas, drones e o helicóptero do Corpo de Bombeiros, e a última

informação sobre sua posição vinha de um aplicativo de atividades físicas registrado antes do desaparecimento [NSC Total, 2026].

No contexto brasileiro, embora o sistema Defesa Civil Alerta tenha avançado com a adoção de tecnologia *Cell Broadcast* para envio de alerta via redes 4G e 5G [Brasil, 2024a], essa abordagem leva avisos das autoridades à população, mas não oferece o caminho inverso, e permanece dependente da infraestrutura de telefonia celular, justamente o recurso mais vulnerável ou ausente nesses cenários. Os recursos de SOS via satélite embarcados em *smartphones* ainda não estão disponíveis no país [Canaltech, 2026], e rádio-faróis pessoais e comunicadores satelitais exigem equipamentos dedicados de custo elevado.

Diante dessa problemática, torna-se evidente a necessidade de soluções de comunicação alternativas, capazes de operar de forma independente da infraestrutura convencional. Nesse contexto, destaca-se o LoRa (*Long Range*), uma tecnologia de comunicação sem fio voltada para longo alcance e baixo consumo energético, amplamente utilizada em aplicações de IoT [Augustin et al., 2016, Bäumker et al., 2019, Devalal and Karthikeyan, 2018]. Entretanto, a arquitetura tradicional baseada em LoRaWAN depende da utilização de *gateways* centrais conectados à Internet, o que representa um ponto único de falha em cenários críticos [Dias and Grilo, 2020]. Soluções abertas como o Meshtastic exploram o LoRa para comunicação sem infraestrutura [Suryadevara and Dutta, 2022, Cass, 2024], mas foram concebidas como redes *mesh* de propósito geral, com troca de mensagens entre todos os nós e pareamento por Bluetooth, o que acrescenta complexidade e consumo a um cenário em que a necessidade central é apenas sinalizar a posição e a condição de quem precisa de ajuda. Além disso, na faixa de 915 MHz, a regulamentação brasileira limita o tempo de permanência de cada transmissão em um canal [Agência Nacional de Telecomunicações (Anatel), 2017], de modo que a informação de alerta precisa caber em poucas dezenas de *bytes* sem sacrificar o alcance.

Considerando esses desafios e oportunidades, este trabalho visa desenvolver um sistema de alerta de emergência baseado em tecnologia LoRa, capaz de transmitir periodicamente a localização e o estado de uma pessoa em risco até a equipe de resgate, sem depender de qualquer infraestrutura de comunicação. O sistema é composto por um aplicativo cliente para Android, denominado Beacon, no qual o usuário se cadastra e, ao conectar o dispositivo LoRa pela porta USB-C do celular, pode acionar um alerta de perigo e atualizar seu estado com mensagens curtas. O dispositivo, baseado no módulo Ebyte E77-900M22S [Chengdu Ebyte Electronic Technology Co., Ltd., 2024] operando em 915 MHz, transmite em intervalos configuráveis um pacote de sinalização (*heartbeat*) contendo o identificador do usuário, suas coordenadas GPS, seu estado e uma mensagem curta. Do outro lado, um dispositivo idêntico conectado a um segundo aplicativo, denominado Sniffer e operado por um agente de emergência, recebe esses pacotes e exibe em um mapa a última posição conhecida e o histórico de cada usuário em alerta. O mesmo *firmware* é utilizado nos dois lados, diferenciando-se apenas pelo papel configurado no nó, e a comunicação ocorre de forma direta, em um único salto, sem roteamento entre nós. A ligação entre o módulo e o celular é feita por cabo USB, por meio da biblioteca *usb-serial-for-android* [mik3y e contribuidores, 2024], dispensando o uso de Bluetooth, que exigiria um circuito adicional no dispositivo e aumentaria seu custo e consumo.

Para alcançar o objetivo geral deste projeto foram elencados os seguintes objetivos

específicos:

- Definir o formato do pacote de alerta, contendo identificador do usuário, coordenadas, estado e mensagem curta, dentro do limite de tamanho imposto pelo tempo de transmissão permitido na faixa de 915 MHz.
- Desenvolver um *firmware* único para o módulo LoRa, com papel configurável (Beacon ou Sniffer), transmissão periódica em intervalo configurável e filtragem de pacotes por tipo de tráfego.
- Estabelecer comunicação entre o dispositivo móvel e o módulo LoRa por meio de conexão USB.
- Desenvolver o aplicativo Beacon, para cadastro do usuário, acionamento do alerta e atualização do estado e da mensagem curta.
- Desenvolver o aplicativo Sniffer, para recepção dos alertas e visualização em mapa da posição e do histórico de cada usuário.
- Avaliar o desempenho do sistema em termos de alcance, taxa de recepção dos pacotes e consumo energético para diferentes intervalos de transmissão e configurações de rádio.

Do ponto de vista metodológico, este trabalho configura-se como uma pesquisa aplicada, de caráter exploratório e experimental, com abordagem quantitativa na etapa de avaliação. O desenvolvimento está organizado em quatro etapas. Na primeira, será conduzido um estudo da literatura sobre a tecnologia LoRa, a regulamentação da faixa de 915 MHz no Brasil e soluções voltadas à localização de pessoas em cenários de emergência. A segunda etapa compreende o desenvolvimento do *firmware*, englobando a definição do formato do pacote, a implementação dos papéis Beacon e Sniffer e da transmissão periódica, com validações progressivas em bancada. Na terceira etapa, serão implementados os aplicativos Beacon e Sniffer, que se comunicam com o módulo LoRa via conexão USB. A quarta e última etapa consiste na realização de testes de campo, nos quais serão mensurados o alcance, a taxa de recepção dos pacotes e o consumo energético, viabilizando uma análise quantitativa do desempenho do sistema.

O restante do documento está organizado da seguinte forma: a Seção 2 aborda o referencial teórico, incluindo conceitos da tecnologia LoRa, sua regulamentação e trabalhos relacionados. A Seção 3 descreve o desenvolvimento do sistema, detalhando a arquitetura, os componentes de *hardware* e *software* e o formato do pacote de alerta. A Seção 4 apresenta os resultados obtidos nos testes realizados. Por fim, a Seção 5 apresenta as conclusões do trabalho e sugestões para trabalhos futuros.

2. Referencial teórico

Esta seção apresenta os conceitos fundamentais que embasam o trabalho, abordando a tecnologia LoRa e seu protocolo padrão LoRaWAN, o tempo de transmissão dos pacotes e a regulamentação da faixa de 915 MHz no Brasil, as topologias de comunicação aplicáveis a cenários sem infraestrutura, a integração de módulos LoRa a *smartphones*, os aspectos de privacidade e integridade das mensagens de alerta e as particularidades da comunicação em cenários de emergência. Ao final, são discutidos trabalhos relacionados que aplicam essas tecnologias em contextos similares ao proposto.

2.1. LoRa, LoRaWAN e Redes LPWAN

A Internet das Coisas (*Internet of Things* – IoT) refere-se à integração de dispositivos físicos à rede, permitindo a coleta, o processamento e a troca de dados de forma autônoma [Tanenbaum and Wetherall, 2011]. Desde os primeiros sistemas de controle remoto até os atuais sensores embarcados distribuídos em larga escala, a IoT expandiu-se para setores como agricultura, saúde, cidades inteligentes e gestão de emergências, conforme ilustrado na Figura 1 [Augustin et al., 2016].

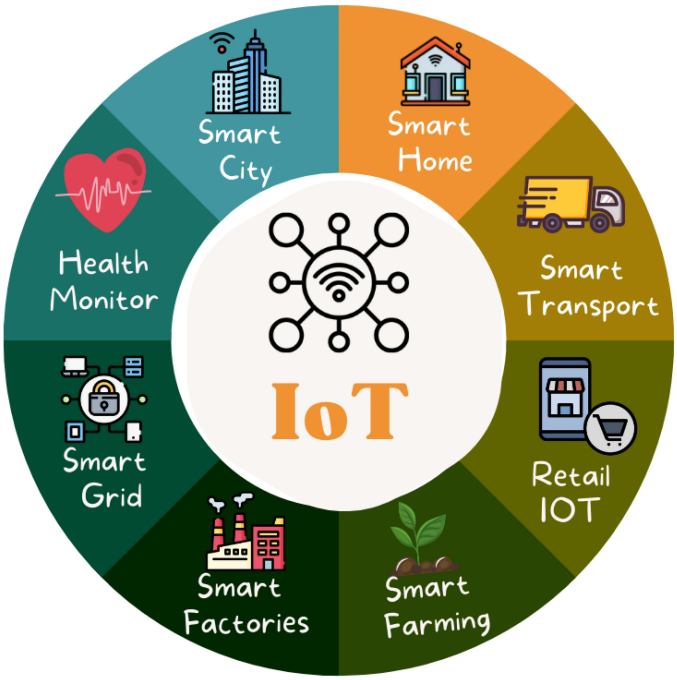


Figura 1. Principais áreas de aplicação da IoT: cidades inteligentes, monitoramento de saúde, transporte, agricultura, indústria, entre outras.

Fonte: adaptado de [Augustin et al., 2016]

Uma das principais demandas de aplicações IoT é a comunicação em áreas de difícil acesso ou sem cobertura de redes celulares convencionais. Para atender a esse requisito, surgiram as redes *Low-Power Wide-Area Networks* (LPWAN), projetadas para oferecer grande alcance com baixo consumo energético e baixas taxas de transmissão de dados [Augustin et al., 2016]. Entre as tecnologias LPWAN disponíveis, como Sigfox[1] e NB-IoT, o LoRa (*Long Range*) se destaca por combinar longo alcance, resistência a interferências e custo reduzido de implantação [Devalal and Karthikeyan, 2018]. Diferentemente do NB-IoT, que opera sobre a infraestrutura das operadoras de telefonia celular, o LoRa utiliza faixas de frequência não licenciadas e pode ser empregado em enlaces diretos entre dispositivos, sem depender de qualquer rede de terceiros. A Figura 2 apresenta um comparativo entre essas tecnologias em termos de alcance, taxa de dados e consumo energético.

[1] Após recuperação judicial em 2022 e venda da empresa, a tecnologia passou a se chamar Sigfox 0G.

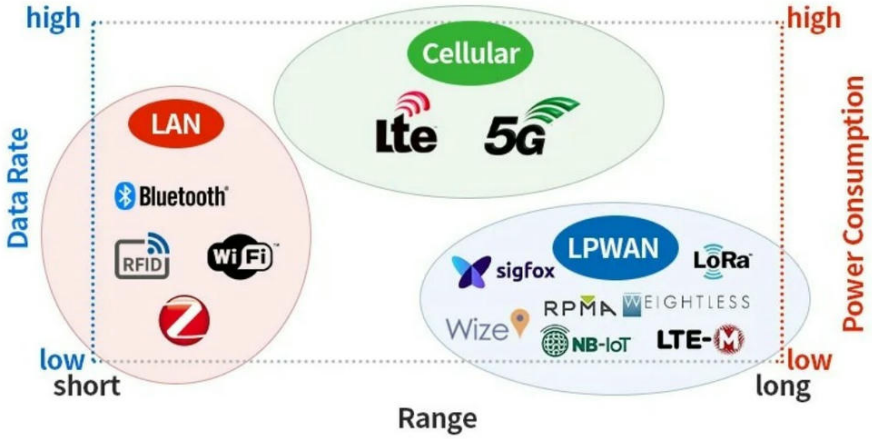


Figura 2. Comparativo entre tecnologias de comunicação sem fio por alcance, taxa de dados e consumo energético: redes de curto alcance (LAN, Wi-Fi, Bluetooth), redes celulares (LTE, 5G) e redes LPWAN (LoRa, Sigfox, NB-IoT, LTE-M).

Fonte: [Devalal and Karthikeyan, 2018]

O LoRa é uma tecnologia de modulação de radiofrequência baseada na técnica *Chirp Spread Spectrum* (CSS), originalmente utilizada em comunicações militares e espaciais [Semtech Corporation, 2019]. A modulação CSS espalha o sinal por uma faixa de frequência mais larga do que o estritamente necessário, conferindo maior imunidade a interferências e a desvanecimentos do canal, além de permitir a decodificação de sinais com relação sinal-ruído negativa [Augustin et al., 2016].

O desempenho do LoRa é regulado principalmente pelo *Spreading Factor* (SF), que determina a taxa de transmissão e o alcance: valores maiores de SF resultam em maior alcance e maior robustez, ao custo de menor taxa de dados e maior tempo de ocupação do canal [Augustin et al., 2016, Bäumker et al., 2019]. Os demais parâmetros configuráveis incluem a largura de banda (*Bandwidth*), a taxa de codificação (*Coding Rate*) e a potência de transmissão, todos com impacto direto no consumo energético e no alcance [Bäumker et al., 2019]. Na faixa de 915 MHz, utilizada no Brasil, o LoRa pode alcançar distâncias de até 15 km em ambientes rurais com linha de visada [Devalal and Karthikeyan, 2018].

O LoRaWAN é o protocolo de rede padronizado pela LoRa Alliance que emprega a modulação LoRa na camada física e define a arquitetura de rede, os procedimentos de acesso ao meio e os mecanismos de segurança [LoRa Alliance, 2017]. A Figura 3 apresenta a pilha de protocolos LoRa/LoRaWAN, evidenciando a separação entre a camada física e as camadas superiores. Sua arquitetura segue uma topologia em estrela de estrelas: dispositivos finais comunicam-se com *gateways* fixos, que repassam os dados a um servidor de rede central via Internet, conforme ilustrado na Figura 4 [LoRa Alliance, 2017, Dias and Grilo, 2020].

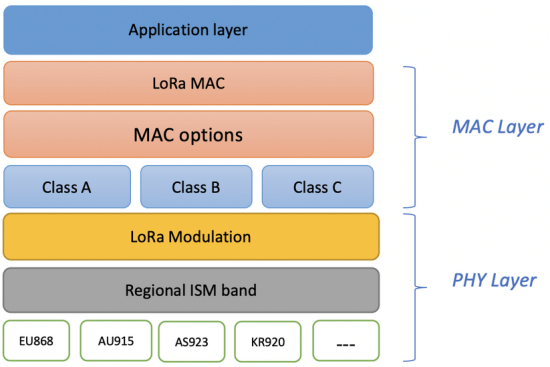


Figura 3. Pilha de protocolos LoRa/LoRaWAN: camada física (PHY) composta pela modulação LoRa e banda ISM regional, e camada de enlace (MAC) com as classes A, B e C e a camada de aplicação.

Fonte: [LoRa Alliance, 2017]

Embora essa arquitetura seja adequada para aplicações de monitoramento em larga escala, ela apresenta limitações importantes para cenários de emergência. A dependência de *gateways* fixos e de conectividade com a Internet cria um ponto único de falha: caso o *gateway* ou o enlace de retorno seja comprometido, toda a comunicação da área coberta é interrompida [Dias and Grilo, 2020]. Além disso, o protocolo LoRaWAN não prevê nativamente a comunicação direta entre dispositivos finais [Dias and Grilo, 2020, Wong et al., 2024]. Por essas razões, o presente trabalho utiliza a modulação LoRa diretamente, em modo ponto a ponto, sem a camada LoRaWAN, de modo que o receptor operado pela equipe de resgate desempenha o papel que caberia ao *gateway*, porém de forma portátil e sem depender da Internet.

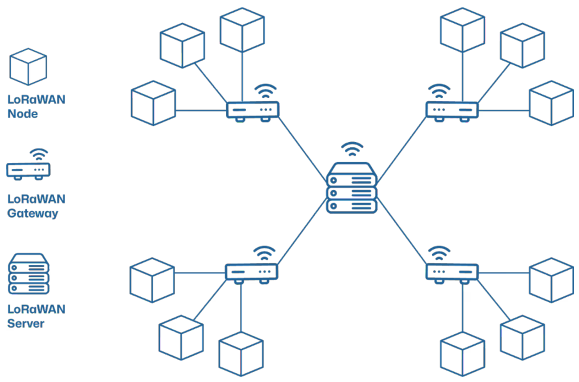


Figura 4. Topologia em estrela de estrelas do LoRaWAN: nós finais comunicam-se com *gateways* via LoRa, que repassam os dados ao servidor de rede central.

Fonte: [LoRa Alliance, 2017]

2.2. Tempo de Transmissão e Regulamentação da Faixa de 915 MHz

Em sistemas LoRa, o tempo que um pacote ocupa o canal (*Time on Air* – ToA) depende do SF, da largura de banda, da taxa de codificação e do tamanho da carga útil. A duração

de um símbolo é dada por $T_{sym} = 2^{SF}/BW$, e o tempo total do pacote corresponde à soma da duração do preâmbulo com a da carga útil [Semtech Corporation, 2019]:

$$T_{pacote} = (n_{pre} + 4{,}25)\, T_{sym} + n_{carga}\, T_{sym} \tag{1}$$

$$n_{carga} = 8 + \max\left(\left\lceil \frac{8PL - 4SF + 28 + 16CRC - 20H}{4(SF - 2DE)} \right\rceil (CR + 4),\ 0\right) \tag{2}$$

em que n_{pre} é o número de símbolos do preâmbulo, n_{carga} é o número de símbolos da carga útil, PL é o tamanho da carga útil em *bytes*, CRC indica a presença do código de verificação (1 ou 0), H indica o uso de cabeçalho implícito (1) ou explícito (0), DE indica a otimização para baixas taxas de dados, habilitada para SF11 e SF12 com largura de banda de 125 kHz, e CR corresponde à taxa de codificação, de 1 (4/5) a 4 (4/8) [Semtech Corporation, 2019]. Como a duração do símbolo dobra a cada incremento do SF, o tempo no ar cresce exponencialmente com o alcance desejado.

No Brasil, os equipamentos LoRa operam como equipamentos de radiação restrita nas faixas de 902 a 907,5 MHz e de 915 a 928 MHz, regulamentadas pela Anatel. Para sistemas que empregam salto de frequência, o Ato nº 14448 estabelece um número mínimo de canais e limita o tempo médio de ocupação de cada canal (*dwell time*) a 400 ms dentro do período de observação definido na norma [Agência Nacional de Telecomunicações (Anatel), 2017]. Em 2025, o Ato nº 14158 revogou a isenção que permitia certificar equipamentos LoRa como sistemas de modulação digital sem a largura de banda mínima de 500 kHz, com período de transição até 5 de outubro de 2026 [Agência Nacional de Telecomunicações (Anatel), 2025, 2026a]. Assim, novos equipamentos LoRa de 125 kHz devem operar com salto de frequência e respeitar o limite de 400 ms por transmissão.

Esse limite restringe diretamente o tamanho das mensagens. No plano regional AU915, adotado no Brasil para o LoRaWAN, a carga útil máxima nas taxas mais baixas é de 51 *bytes*, sendo reduzida para 11 *bytes* quando a restrição de *dwell time* está ativa [LoRa Alliance, 2022]. Em enlaces LoRa ponto a ponto, sem a sobrecarga do LoRaWAN, a Tabela 1 apresenta o tempo no ar calculado pelas Equações 1 e 2 para diferentes valores de SF e o maior pacote que respeita o limite de 400 ms. Observa-se que SF11 e SF12, que oferecem o maior alcance, não comportam nenhum pacote útil dentro desse limite, e que em SF10 a carga útil fica restrita a 24 *bytes*.

Desse modo, o projeto de um sistema de alerta baseado em LoRa envolve um compromisso entre alcance e quantidade de informação por pacote, o que exige uma codificação compacta dos dados de localização e de estado do usuário.

2.3. Topologias de Comunicação: Enlace Direto e Redes *Mesh*

Redes *mesh* são redes sem fio nas quais cada nó pode atuar simultaneamente como terminal e como roteador, encaminhando mensagens destinadas a outros nós [Murthy and Manoj, 2004]. Essa arquitetura, também denominada rede *ad hoc*, dispensa infraestrutura centralizada e permite a comunicação por múltiplos saltos (*multi-hop*), ampliando a cobertura total da rede [Murthy and Manoj, 2004, Perkins, 2001]. Combinada ao LoRa,

Tabela 1. Tempo no ar de pacotes LoRa e maior carga útil que respeita o limite de 400 ms (BW = 125 kHz, CR = 4/5, preâmbulo de 8 símbolos, cabeçalho explícito e CRC habilitado; B = *bytes*).

SF	ToA com 20 B (ms)	ToA com 51 B (ms)	Carga máx. em 400 ms (B)
7	57	103	255
8	103	185	138
9	185	329	66
10	371	616	24
11	741	1315	–
12	1319	2466	–

Fonte: elaborado pelos autores a partir de [Semtech Corporation, 2019]

a topologia *mesh* elimina a dependência de *gateways* fixos, e sua viabilidade técnica tem sido demonstrada em diversos estudos [Berto et al., 2021, Wong et al., 2024].

Essa flexibilidade, contudo, tem custos. A descoberta e a manutenção de rotas exigem a troca de mensagens de controle, e cada salto adicional retransmite o mesmo pacote, multiplicando a ocupação do canal, o risco de colisões e o consumo energético dos nós intermediários [Wong et al., 2024, Dias and Grilo, 2020]. Em protocolos reativos como o AODV, a descoberta de rotas é feita por inundação controlada da rede, o que se torna custoso diante das restrições de tempo de transmissão discutidas na Seção 2.2 [Perkins et al., 2003, Lundell et al., 2018].

Na comunicação por enlace direto (um único salto), por outro lado, o transmissor envia o pacote em difusão e qualquer receptor ao alcance o recebe, sem roteamento nem retransmissão. Essa topologia limita a cobertura ao alcance de um único enlace, mas reduz a complexidade do *firmware*, o tempo de ocupação do canal e o consumo energético. Para a aplicação proposta, em que o fluxo de informação é unidirecional, da vítima para a equipe de resgate, e a equipe pode se deslocar em direção à área de busca, o enlace direto atende à necessidade central do sistema com menor custo. A extensão da cobertura por meio de retransmissores permanece como possibilidade para trabalhos futuros.

2.4. Integração de Módulos LoRa a *Smartphones*

Os *smartphones* não possuem rádio LoRa, de modo que sua utilização em sistemas LoRa depende de um módulo externo. Essa integração pode ser realizada por *Bluetooth Low Energy* (BLE), Wi-Fi ou conexão física. Soluções como o LOCATE e o Meshtastic adotam o BLE, que dispensa cabos, mas exige um rádio adicional no dispositivo e um processo de pareamento [Ferretti et al., 2019, Suryadevara and Dutta, 2022]. Höchst et al. [2020] destacam os desafios de integrar o *hardware* LoRa a dispositivos móveis convencionais, entre eles o consumo de energia e a complexidade de configuração por usuários leigos.

O SoC STM32WLE5, utilizado no módulo Ebyte E77-900M22S, integra em um único componente um microcontrolador ARM Cortex-M4 e um transceptor sub-GHz compatível com a modulação LoRa, mas não possui rádio Bluetooth [STMicroelectronics, 2024, Chengdu Ebyte Electronic Technology Co., Ltd., 2024]. A adoção de BLE

exigiria, portanto, um segundo circuito integrado, com aumento de custo, de área de placa e de consumo. Em contrapartida, a conexão física por USB aproveita o conversor USB–serial presente na placa de avaliação [Chengdu Ebyte Electronic Technology Co., Ltd., 2022, Nanjing Qinheng Microelectronics Co., Ltd. (WCH), 2018], dispensa pareamento e permite que o módulo seja alimentado pela bateria do próprio celular.

No sistema Android, essa comunicação é viabilizada pelo modo USB *host*, no qual o *smartphone* atua como hospedeiro e fornece alimentação ao periférico conectado [Google, 2024b]. A biblioteca de código aberto *usb-serial-for-android* implementa, sobre essa interface, *drivers* para os principais conversores USB–serial, incluindo a família CH340, permitindo que o aplicativo troque dados com o módulo como em uma porta serial convencional [mik3y e contribuidores, 2024].

2.5. Privacidade e Integridade das Mensagens de Alerta

A comunicação por radiofrequência em canais abertos expõe os dados transmitidos a ameaças como interceptação, injeção de mensagens falsas e ataques de repetição (*replay attacks*), nos quais um adversário captura um pacote legítimo e o retransmite posteriormente [Tanenbaum and Wetherall, 2011]. No LoRaWAN, essas ameaças são tratadas com criptografia simétrica AES-128, por meio de chaves de sessão de rede e de aplicação gerenciadas por um servidor central [LoRa Alliance, 2017], modelo que não se aplica a enlaces ponto a ponto sem infraestrutura [Dias and Grilo, 2020]. Nesses casos, a alternativa mais comum é a criptografia simétrica com chave pré-compartilhada, como o AES no modo *Counter with CBC-MAC* (CCM), que cifra e autentica cada mensagem [Whiting et al., 2003, Dworkin, 2004].

Em um sistema de alerta, entretanto, esses mecanismos competem diretamente com o limite de tamanho do pacote discutido na Seção 2.2: a etiqueta de autenticação do CCM acrescenta de 4 a 16 *bytes* a cada mensagem [Whiting et al., 2003], o que pode inviabilizar a operação nos fatores de espalhamento de maior alcance. Por outro lado, mecanismos leves, como um contador de sequência incremental em cada pacote, permitem ao receptor descartar duplicatas e identificar retransmissões [Murthy and Manoj, 2004].

Do ponto de vista da privacidade, a Lei Geral de Proteção de Dados Pessoais (LGPD) estabelece princípios como a necessidade e a minimização no tratamento de dados pessoais [Brasil, 2018]. Assim, dados identificadores sensíveis, como o CPF utilizado no cadastro, não devem ser transmitidos pelo canal de rádio; em seu lugar, o pacote de alerta carrega apenas um identificador curto, cuja associação ao usuário fica restrita aos sistemas da equipe de resgate.

2.6. Comunicação e Localização de Vítimas em Cenários de Emergência

A manutenção da comunicação durante e após eventos de desastre é um requisito crítico para a coordenação das operações de resposta [Haddow and Haddow, 2014]. Estudos sobre o impacto de desastres na infraestrutura de telecomunicações mostram que redes celulares e de fibra óptica estão entre os serviços mais vulneráveis, tanto por danos físicos diretos quanto por falta de energia elétrica [El Khaled and Mcheick, 2019, Goos et al., 2024]. A ausência de comunicação compromete desde a mobilização de equipes de resgate até o envio de alertas à população em risco [Haddow and Haddow, 2014, Chang et al., 2007].

Nas operações de busca e resgate, a informação mais valiosa é a localização da vítima e sua condição atual: quanto mais precisa e recente for essa informação, menor a área de busca e o tempo de resposta [Pine, 2018]. Os sistemas públicos de alerta, como o Defesa Civil Alerta, foram concebidos para o fluxo inverso, das autoridades para a população [Brasil, 2024a], e os recursos de SOS via satélite embarcados em *smartphones* ainda não estão disponíveis no Brasil [Canaltech, 2026].

Nesse contexto, tecnologias que operam de forma independente da infraestrutura convencional têm valor estratégico [Pine, 2018]. Rádios de ondas curtas, redes satelitais e sistemas *trunking* digitais são alternativas tradicionais, mas apresentam limitações em termos de custo, complexidade operacional ou dependência de equipamentos especializados [El Khaled and Mcheick, 2019]. Dispositivos de baixo custo, baseados em componentes de fácil aquisição e operáveis por usuários com treinamento mínimo, representam uma alternativa complementar com potencial de adoção ampla [Pine, 2018, Macaraeg et al., 2021].

2.7. Trabalhos Relacionados

Diversos trabalhos investigaram a aplicação do LoRa à comunicação em situações de emergência e ambientes sem infraestrutura. Macaraeg et al. [2021] implementaram uma rede *mesh* LoRa com protocolo AODV modificado voltada à comunicação de emergência *off-grid*, avaliando o desempenho em termos de taxa de entrega e latência em diferentes configurações de rede. Os resultados indicam que o protocolo adaptado é viável para redes de pequeno e médio porte em ambientes abertos.

Berto et al. [2021] propõem uma rede LoRa *mesh* para comunicação ponto a ponto de longo alcance, com foco em um protocolo de roteamento leve e tolerante a falhas de enlace. Os autores avaliam o sistema em testes de campo e demonstram que a topologia *mesh* estende significativamente o alcance efetivo da rede em relação a enlaces ponto a ponto simples.

O sistema LOCATE [Ferretti et al., 2019] integra transceptores LoRa a dispositivos móveis via BLE para disseminação de alertas de emergência em cenários pós-desastre. O sistema foi projetado para operar sem infraestrutura de rede e demonstrou capacidade de cobertura de áreas urbanas com um número reduzido de nós.

Höchst et al. [2020] desenvolveram um sistema de comunicação dispositivo a dispositivo baseado em LoRa para *smartphones*, voltado a cenários de crise. O trabalho destaca os desafios de integrar o *hardware* LoRa a dispositivos móveis convencionais e avalia o desempenho do sistema em simulações de cenários de desastre.

No âmbito das plataformas abertas, o Meshtastic [Suryadevara and Dutta, 2022, Cass, 2024] é um *firmware* de código aberto que implementa uma rede *mesh* LoRa com suporte a mensagens de texto e coordenadas GPS. A plataforma tem sido adotada em aplicações de comunicação *off-grid* por grupos de radioamadores, voluntários de busca e resgate e equipes de operações em campo, demonstrando a praticidade da abordagem em situações reais.

Em conjunto, esses trabalhos demonstram a viabilidade do LoRa para comunicação em ambientes sem infraestrutura, mas concentram-se na troca de mensagens entre participantes de uma rede, em geral com topologia *mesh* e integração por BLE. O presente

trabalho se diferencia por tratar um problema mais restrito: a sinalização periódica da posição e do estado de uma pessoa em risco para um receptor dedicado, operado pela equipe de resgate, que apresenta essas informações em um mapa. Para isso, adota um enlace direto de um único salto, um pacote compacto dimensionado para os limites regulatórios da faixa de 915 MHz e a conexão do módulo ao *smartphone* Android via USB, utilizando um componente que integra microcontrolador e transceptor LoRa (Ebyte E77). O Quadro 1 resume as principais características dos trabalhos relacionados em comparação com a solução proposta.

Trabalho	**Topologia**	**Finalidade**	**Interface c/ usuário**	**Nós móveis**	**Coord. GPS**
[Macaraeg et al., 2021]	Mesh	Troca de mensagens	—	Não	Não
[Berto et al., 2021]	Mesh	Troca de mensagens	—	Não	Não
LOCATE [Ferretti et al., 2019]	Mesh	Disseminação de alertas	Smartphone (BLE)	Sim	Sim
[Höchst et al., 2020]	P2P	Troca de mensagens	Smartphone (BLE)	Sim	Não
Meshtastic [Suryadevara and Dutta, 2022, Cass, 2024]	Mesh	Mensagens e posição	App móvel (BLE/Wi-Fi)	Sim	Sim
Este trabalho	**Enlace direto**	**Alerta de posição e estado**	**App Android (USB)**	**Sim**	**Sim**

Quadro 1. Quadro comparativo entre os trabalhos relacionados e a solução proposta.

3. Documentação técnica da proposta

Esta seção apresenta o detalhamento técnico do sistema proposto, organizado em três camadas: o aplicativo Beacon, executado no *smartphone* do usuário em situação de risco; o nó LoRa, responsável pela transmissão e pela recepção dos alertas por radiofrequência; e o aplicativo Sniffer, executado no *smartphone* do agente de emergência. A Subseção 3.1 descreve a arquitetura geral e o fluxo de informação; a Subseção 3.2 apresenta os componentes físicos do nó; a Subseção 3.3 descreve o *firmware* embarcado, incluindo os papéis do nó, o formato do pacote de alerta e o protocolo de comunicação com o aplicativo; e a Subseção 3.4 descreve os aplicativos Beacon e Sniffer.

3.1. Arquitetura geral

O sistema é composto por dois conjuntos idênticos de *hardware*, cada um formado por um nó LoRa conectado por cabo USB a um *smartphone* Android, conforme ilustra a Figura 5. Do lado do usuário, o aplicativo Beacon fornece ao nó a posição, o estado e a mensagem curta do usuário, e o nó transmite periodicamente essas informações em um pacote de alerta (*heartbeat*). Do lado da equipe de resgate, um nó configurado como receptor entrega os alertas recebidos ao aplicativo Sniffer, que os apresenta em um mapa.

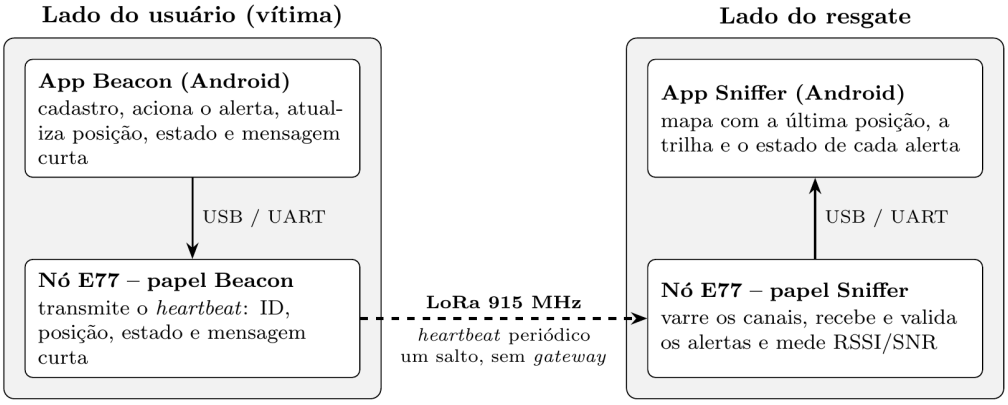


Figura 5. Arquitetura do sistema: o nó do usuário e o nó do resgate utilizam o mesmo *hardware* e o mesmo *firmware*, diferenciando-se apenas pelo papel configurado. A comunicação entre eles ocorre em um único salto LoRa, sem *gateway* nem infraestrutura de rede.

Fonte: elaborado pelos autores.

O fluxo de informação é unidirecional: o alerta parte da vítima e chega à equipe de resgate, sem confirmação de recebimento pelo rádio. Essa escolha decorre das restrições discutidas na Seção 2.2: cada confirmação ocuparia o canal e consumiria energia, e a transmissão periódica já cumpre o papel de redundância, uma vez que um alerta perdido é substituído pelo seguinte. Conforme justificado na Seção 2.3, não há roteamento nem retransmissão entre nós; a cobertura corresponde ao alcance de um único enlace LoRa, e o receptor, por ser portátil, pode ser deslocado em direção à área de busca.

3.2. Hardware

O *hardware* de cada nó foi construído a partir da placa de avaliação Ebyte E77-900MBL-01, pertencente à linha de *kits* de avaliação MBL da fabricante, concebidos para a prototipagem rápida com seus módulos sem fio sub-GHz [Chengdu Ebyte Electronic Technology Co., Ltd., 2022]. A placa reúne, em uma única plataforma, o módulo de rádio, o conversor de interface USB e os circuitos auxiliares de alimentação, depuração e medição, levando a maior parte dos sinais do módulo a dois conectores de 14 pinos posicionados nas bordas [Chengdu Ebyte Electronic Technology Co., Ltd., 2022]. O mesmo *hardware* é utilizado nos dois lados do sistema, o que simplifica a produção, os testes e a manutenção. A Figura 6 apresenta a placa utilizada, com a identificação de seus principais elementos. As subseções a seguir descrevem, na ordem do fluxo de dados, o módulo E77-900M22S e seu SoC (Subseção 3.2.1), o subsistema de rádio LoRa (Subseção 3.2.2), a interface USB–UART com o dispositivo Android (Subseção 3.2.3) e a alimentação e a medição de consumo (Subseção 3.2.4).

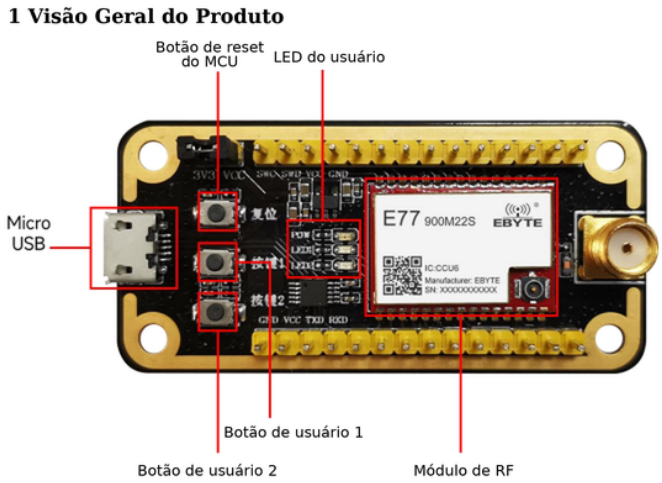


Figura 6. Placa de avaliação E77-900MBL-01, com destaque para o módulo de rádio, o conector Micro-USB, os botões de *reset* e de usuário, os LEDs indicadores e o conector de antena.

Fonte: adaptado de [Chengdu Ebyte Electronic Technology Co., Ltd., 2022].

3.2.1. Módulo E77-900M22S e o SoC STM32WLE5CCU6

O elemento central de cada nó é o módulo E77-900M22S, um módulo SMD que encapsula o *System-on-Chip* (SoC) STM32WLE5CCU6, os cristais de referência (32,768 kHz e 32 MHz), a rede de adaptação de radiofrequência e o circuito de chaveamento de antena em um único componente de 14 × 20 × 2,7 mm [Chengdu Ebyte Electronic Technology Co., Ltd., 2024].

O STM32WLE5CCU6, da STMicroelectronics, é um microcontrolador baseado em um núcleo ARM Cortex-M4 a 48 MHz, com 256 KB de memória *flash* e 64 KB de SRAM [STMicroelectronics, 2024]. No mesmo encapsulamento de silício, o SoC integra um subsistema de rádio sub-GHz, descrito na Subseção 3.2.2, e periféricos de apoio, como um gerador de números aleatórios por *hardware* (*True Random Number Generator* – TRNG). O Quadro 2 resume as principais características do módulo e do SoC.

3.2.2. Subsistema de rádio LoRa e integração com o controlador

A principal vantagem arquitetural do STM32WLE5CCU6 reside na integração, em um único encapsulamento de silício, do núcleo de processamento e do subsistema de rádio sub-GHz, este último baseado na propriedade intelectual do transceptor Semtech SX126x [Semtech Corporation, 2019]. Diferentemente de soluções que acoplam um microcontrolador a um transceptor LoRa externo por meio de um barramento SPI exposto em placa, no STM32WLE5 essa comunicação ocorre internamente: o núcleo Cortex-M4 controla o rádio por meio de um barramento SPI dedicado e interno (*SUBGHZSPI*), acompanhado das linhas de *reset*, de ocupação (*busy*) e de interrupção do rádio [STMicroelectronics, 2024]. Essa integração reduz a contagem de componentes, a área de placa e os pontos de

Característica	Especificação
SoC	STM32WLE5CCU6 (ARM Cortex-M4 a 48 MHz)
Memória	256 KB *flash*, 64 KB SRAM
Rádio	Sub-GHz integrado (IP Semtech SX126x); modulações LoRa, (G)FSK, (G)MSK e BPSK
Faixa de frequência	868–930 MHz (915 MHz no Brasil)
Potência de transmissão	Programável, até +22 dBm
Sensibilidade	Até -148 dBm (LoRa, SF12)
Segurança em *hardware*	Acelerador AES 128/256 bits, TRNG e PKA
Interfaces	UART, SPI, I^2C, GPIO, ADC
Alimentação	1,8–3,6 V (desempenho ótimo $\geq$ 3,3 V)
Dimensões do módulo	$14 \times 20 \times 2{,}7$ mm

Quadro 2. Principais características do módulo E77-900M22S e do SoC STM32WLE5CCU6.

falha, fatores relevantes para um dispositivo destinado a operar em campo.

O rádio permite a configuração dos parâmetros de *Spreading Factor*, largura de banda, taxa de codificação e potência de transmissão, que determinam o compromisso entre alcance e tempo no ar discutido na Seção 2.2. O módulo E77-900M22S expõe esse rádio na faixa de 868 a 930 MHz, que abrange a faixa de 915 MHz adotada no Brasil, com potência de transmissão programável de até +22 dBm [Chengdu Ebyte Electronic Technology Co., Ltd., 2024]. O encaminhamento do sinal entre os caminhos de transmissão e de recepção é realizado por uma chave de RF interna ao módulo, comandada por dois pinos do microcontrolador (`RF_TXEN` e `RF_RXEN`), enquanto o casamento de impedância e a conexão à antena são providos pela rede de adaptação da placa de avaliação (indutor `L1` e conector `ANT1`) [Chengdu Ebyte Electronic Technology Co., Ltd., 2024]. O transceptor oferece ainda, nativamente, o recurso de detecção de atividade de canal (*Channel Activity Detection* – CAD), empregado pelo *firmware* tanto para verificar se o canal está livre antes de transmitir quanto para a varredura de canais no receptor, conforme descrito na Subseção 3.3.4.

3.2.3. Interface USB–UART e conexão com o dispositivo Android

A comunicação entre o nó e o dispositivo móvel é estabelecida por meio do conector Micro-USB da placa. Como o STM32WLE5CCU6 não dispõe de controlador USB nativo, a placa emprega o circuito integrado CH340X, que atua como conversor USB–UART [Nanjing Qinheng Microelectronics Co., Ltd. (WCH), 2018]: do lado do hospedeiro, o CH340X enumera-se como um dispositivo serial; do lado do módulo, conecta-se à porta serial do STM32WLE5, levada aos pinos 9 (TXD) e 10 (RXD) dos conectores da placa [Chengdu Ebyte Electronic Technology Co., Ltd., 2022]. O aplicativo Android, operando em modo USB *host* [Google, 2024b], acessa o conversor como uma porta serial virtual e troca com o *firmware* os quadros descritos na Subseção 3.3.5. Como o STM32WLE5 não possui rádio Bluetooth, essa conexão física dispensa um segundo circuito integrado e o

processo de pareamento, conforme discutido na Seção 2.4.

Dessa forma, o caminho completo de um alerta originado no aplicativo Beacon até o meio físico percorre a sequência: aplicativo Android → USB → CH340X → UART → *firmware* no Cortex-M4 → *SUBGHZSPI* → rádio SX126x → antena, sendo o percurso inverso seguido no nó receptor até o aplicativo Sniffer, conforme ilustra a Figura 7.

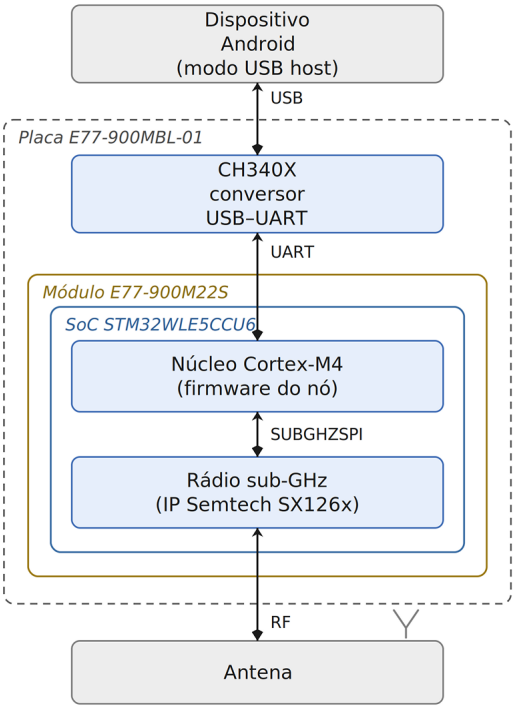


Figura 7. Caminho de dados de um nó: do aplicativo Android, via USB e conversor CH340X, à porta serial do STM32WLE5 e, deste, via barramento interno *SUBGHZSPI*, ao rádio sub-GHz e à antena. No nó receptor, o mesmo caminho é percorrido em sentido inverso.

Fonte: elaborado pelos autores.

3.2.4. Alimentação e medição de consumo

A placa é alimentada pelos 5 V do barramento USB (`VBUS`), fornecidos pelo próprio *smartphone* em modo USB *host*, e regulados para 3,3 V por um regulador linear de baixa queda (*Low-Dropout* – LDO) ME6211 [Nanjing Micro One Electronics Inc., 2017], tensão na qual o módulo E77-900M22S apresenta seu melhor desempenho, dentro de uma faixa de operação de 1,8 a 3,6 V [Chengdu Ebyte Electronic Technology Co., Ltd., 2024]. Dessa forma, o nó não possui bateria própria, e toda a energia consumida pelo sistema provém da bateria do celular, o que torna o consumo do nó um fator determinante para a autonomia do usuário em situação de risco.

Um recurso particularmente relevante para os objetivos deste trabalho é o *jumper* de medição de corrente, identificado no esquema como o conector de dois pinos `3V3–VCC`. Em operação normal, um conector curto-circuita esses pinos, ligando a ali-

mentação de 3,3 V ao pino `VCC` do módulo. Ao remover esse conector e inserir um amperímetro em série, isola-se a corrente consumida exclusivamente pelo módulo de rádio do restante da placa [Chengdu Ebyte Electronic Technology Co., Ltd., 2022]. Esse arranjo será utilizado na caracterização do consumo energético descrita na Seção 4, complementado pela medição do consumo total da placa no barramento USB, que inclui o conversor CH340X e o regulador.

3.3. Firmware

O *firmware* é único para todos os nós e foi desenvolvido para o núcleo Cortex-M4 do STM32WLE5CCU6. Suas responsabilidades são configurar o rádio, montar e transmitir os pacotes de alerta, receber e filtrar os pacotes no papel de receptor e trocar comandos e dados com o aplicativo pela porta serial. As subseções a seguir descrevem os papéis do nó, o formato do pacote de alerta, a política de transmissão, o uso dos canais e o protocolo serial com o aplicativo.

3.3.1. Papéis do nó

O comportamento de cada nó é definido por um parâmetro de papel (*role*), configurado pelo aplicativo no momento da conexão e persistido na memória *flash*. São definidos dois papéis:

- **Beacon:** o nó recebe do aplicativo a posição, o estado e a mensagem curta do usuário e, enquanto o alerta estiver ativo, transmite periodicamente o pacote de alerta. Entre as transmissões, o rádio permanece desligado e o microcontrolador entra em modo de baixo consumo. Pacotes de alerta recebidos de outros usuários são ignorados.
- **Sniffer:** o nó mantém o rádio em recepção, valida cada pacote recebido e repassa ao aplicativo os alertas, acompanhados da intensidade do sinal recebido (*Received Signal Strength Indicator* – RSSI) e da relação sinal-ruído (*Signal-to-Noise Ratio* – SNR) medidas pelo rádio.

A distinção entre os tipos de tráfego é feita por um campo de porta no início de cada pacote, análogo ao conceito de porta das camadas de transporte [Tanenbaum and Wetherall, 2011]. O nó no papel Beacon descarta os pacotes da porta de alerta, de modo que um usuário em situação de risco não tenha sua bateria consumida pelo processamento de alertas de terceiros, enquanto o nó no papel Sniffer não pode ignorá-los. Valores de porta adicionais são reservados para extensões futuras, como mensagens de controle ou de teste, sem alteração do formato básico do pacote.

3.3.2. Formato do pacote de alerta

O pacote de alerta foi projetado para caber no limite de 400 ms de tempo no ar nos fatores de espalhamento de maior alcance admissíveis, conforme a Tabela 1. Todos os campos são serializados explicitamente em *big-endian*, evitando problemas de alinhamento e de preenchimento (*padding*) de estruturas entre diferentes compiladores e arquiteturas. O Quadro 3 descreve a estrutura do pacote.

Campo	Bytes	Descrição
`port`	1	Tipo de tráfego (porta de alerta)
`user_id`	4	Identificador do usuário, gerado no cadastro
`seq`	2	Número de sequência do alerta, incrementado a cada transmissão
`lat`	4	Latitude, inteiro com sinal em unidades de 10^{-7} grau
`lon`	4	Longitude, inteiro com sinal em unidades de 10^{-7} grau
`status`	1	Código do estado do usuário (Quadro 4)
`battery`	1	Nível de bateria do *smartphone*, em percentual
`msg`	0 a N	Mensagem curta em texto livre (UTF-8)

Quadro 3. Estrutura do pacote de alerta transmitido pelo nó Beacon.

O cabeçalho fixo ocupa 17 *bytes*. O comprimento da mensagem não é transmitido explicitamente, pois é obtido do campo de comprimento do cabeçalho explícito da camada física LoRa. A representação das coordenadas como inteiros de 32 bits em unidades de 10^{-7} grau oferece resolução da ordem de 1 cm, muito superior à precisão do GPS de um *smartphone*, com metade do espaço de uma representação em ponto flutuante de dupla precisão. O tamanho máximo N da mensagem depende do fator de espalhamento configurado: considerando o limite de 400 ms, restam 7 *bytes* em SF10 e 49 *bytes* em SF9.

Como o espaço para texto livre é reduzido, as situações mais comuns são codificadas no campo `status`, de modo que a informação essencial chegue à equipe de resgate mesmo quando a mensagem estiver vazia. O Quadro 4 apresenta o conjunto inicial de códigos, que pode ser ampliado sem alteração do formato do pacote.

Código	Estado
0	Alerta encerrado pelo usuário (em segurança)
1	Perigo, sem detalhamento
2	Perdido ou desorientado
3	Ferido ou com necessidade de atendimento médico
4	Incêndio próximo
5	Ilhado por água ou deslizamento
6	Sem água ou suprimentos

Quadro 4. Códigos de estado do usuário transmitidos no campo `status`.

3.3.3. Política de transmissão

No papel Beacon, o nó transmite o pacote de alerta em intervalos regulares enquanto o alerta estiver ativo. O intervalo é configurável pelo aplicativo, com valor padrão de cinco minutos, e representa o principal compromisso do sistema entre a atualidade da informação disponível à equipe de resgate e o consumo de energia. Além das transmissões

periódicas, o nó transmite imediatamente sempre que o usuário altera seu estado ou sua mensagem, de modo que mudanças relevantes, como a aproximação de um incêndio, não aguardem o próximo ciclo. Ao encerrar o alerta, o nó transmite um último pacote com o código de estado 0, informando à equipe de resgate que o usuário está em segurança.

O temporizador de transmissão é mantido pelo próprio nó, a partir do relógio de tempo real (RTC) do microcontrolador, e não pelo aplicativo. Assim, o nó continua transmitindo com a última posição conhecida mesmo que o sistema operacional suspenda temporariamente o aplicativo para economizar energia. O aplicativo, por sua vez, atualiza a posição armazenada no nó sempre que obtém uma nova leitura do GPS.

Como vários usuários podem acionar alertas na mesma área, e o acesso ao meio segue um esquema do tipo ALOHA, sem coordenação entre os nós, cada intervalo recebe uma dispersão aleatória (*jitter*) gerada pelo TRNG do SoC [STMicroelectronics, 2024], evitando que dois nós que iniciaram o alerta ao mesmo tempo colidam de forma persistente. Antes de cada transmissão, o nó executa uma verificação de atividade de canal (CAD) e, caso detecte outra transmissão em curso, adia a sua por um intervalo aleatório curto.

3.3.4. Uso dos canais e conformidade regulatória

Conforme discutido na Seção 2.2, equipamentos LoRa com largura de banda de 125 kHz devem empregar salto de frequência e limitar a ocupação de cada canal a 400 ms. Para atender a esse requisito, o *firmware* define um conjunto de canais na faixa de 915 a 928 MHz e seleciona, a cada transmissão, um canal diferente segundo uma sequência pseudoaleatória, de modo que a ocupação média se distribua entre todos os canais. A configuração de rádio (SF, largura de banda e taxa de codificação) e o tamanho máximo da mensagem são escolhidos de forma que nenhum pacote ultrapasse 400 ms no ar.

O salto de frequência impõe um desafio ao receptor: como o nó Sniffer dispõe de um único rádio e não conhece de antemão o canal da próxima transmissão, ele não pode permanecer sintonizado em um único canal. A estratégia adotada é a varredura cíclica dos canais com o recurso CAD, que detecta a presença de um preâmbulo LoRa em poucos símbolos; ao detectar atividade, o receptor permanece no canal e recebe o pacote. Para que a detecção ocorra, o preâmbulo transmitido deve ser mais longo do que o ciclo completo de varredura, o que acrescenta tempo no ar a cada pacote. O equilíbrio entre o número de canais, o comprimento do preâmbulo e o tamanho máximo da mensagem será avaliado experimentalmente na Seção 4.

3.3.5. Protocolo de comunicação serial com o aplicativo

Como a UART entrega um fluxo contínuo de *bytes*, sem delimitação inerente de mensagens, a comunicação entre o nó e o aplicativo emprega um protocolo de enquadramento próprio. Cada quadro inicia com um *byte* delimitador fixo, seguido de um campo de tipo de 1 *byte*, de um campo de comprimento de 1 *byte*, da carga útil e de um código de verificação CRC-16, que permite descartar quadros corrompidos e ressincronizar a leitura a partir do próximo delimitador. O Quadro 5 resume os tipos de quadro definidos.

Quadro	Sentido	Conteúdo
`CONFIG`	App → nó	Papel, identificador do usuário, intervalo de transmissão e parâmetros de rádio
`ALERT_START`	App → nó	Inicia o alerta com a posição, o estado e a mensagem atuais
`UPDATE`	App → nó	Atualiza posição, estado, bateria ou mensagem; mudanças de estado ou de mensagem disparam transmissão imediata
`ALERT_STOP`	App → nó	Encerra o alerta, com transmissão do estado 0
`RX_ALERT`	Nó → app	Alerta recebido (papel Sniffer), acompanhado de RSSI e SNR
`STATUS`	Nó → app	Confirmação de comandos, papel ativo, versão do *firmware* e erros

Quadro 5. Tipos de quadro trocados entre o aplicativo e o nó pela porta serial.

O enlace USB–UART entre o aplicativo e o nó é local e confiável. Na recepção, o nó Sniffer entrega ao aplicativo os campos do pacote já validados, de modo que a interpretação do formato de rádio permaneça concentrada no *firmware*.

3.4. Aplicativos móveis

O sistema conta com dois aplicativos Android, escritos em Kotlin, que compartilham a mesma camada de comunicação com o nó e diferem na interface e no papel configurado. O aplicativo Beacon é utilizado pelo usuário que pode precisar de socorro, e o aplicativo Sniffer, pelo agente de emergência. Ambos comunicam-se com o nó por meio da conexão USB apresentada na Subseção 3.2.3. A Subseção 3.4.1 descreve o aplicativo Beacon, a Subseção 3.4.2 descreve o aplicativo Sniffer e a Subseção 3.4.3 detalha a camada de comunicação serial comum aos dois.

3.4.1. Aplicativo Beacon

No primeiro uso, o usuário realiza um cadastro com seu CPF e dados básicos de identificação. O cadastro é realizado com acesso à Internet, antes da ida a campo, junto a um servidor mantido pela entidade responsável pelas operações de resgate. O servidor gera um identificador aleatório de 32 bits (`user_id`), que é o único dado de identificação transmitido pelo rádio, armazenado pelo aplicativo. O CPF não é transmitido nem derivado diretamente no identificador, uma vez que o espaço de valores do CPF é pequeno o suficiente para que um identificador calculado a partir dele pudesse ser revertido, em consonância com o princípio de minimização da LGPD discutido na Seção 2.5.

Ao conectar o nó, o aplicativo envia o quadro `CONFIG` com o papel Beacon, e o identificador do usuário. A tela principal apresenta o botão de acionamento do alerta, a seleção do estado entre os códigos do Quadro 4, um campo para a mensagem curta, que limita a digitação ao tamanho máximo admitido pela configuração de rádio, e o ajuste do intervalo de transmissão. Durante o alerta, a tela exibe o horário da última transmissão e o estado atual, e o usuário pode alterar o estado ou a mensagem a qualquer momento.

Enquanto o alerta estiver ativo, o aplicativo executa um serviço em primeiro plano (*foreground service*), recurso do Android que mantém a tarefa em execução com uma notificação visível ao usuário, mesmo com a tela desligada [Google, 2024a]. Esse serviço obtém a posição pelos serviços de localização do Android e a envia ao nó por meio do quadro `UPDATE`, juntamente com o nível de bateria. Como o temporizador de transmissão reside no nó (Subseção 3.3.3), uma eventual suspensão do aplicativo apenas deixa de atualizar a posição, sem interromper o envio dos alertas.



Figura 8. Interface do aplicativo Beacon *(em desenvolvimento)*: acionamento do alerta, seleção do estado e mensagem curta.

Fonte: elaborado pelos autores.

3.4.2. Aplicativo Sniffer

O aplicativo Sniffer configura o nó no papel receptor e apresenta ao agente de emergência um mapa com os usuários em alerta. Para cada `user_id` recebido, o aplicativo mantém a última posição conhecida, o histórico de posições, o estado, a mensagem, o nível de bateria e o horário do último contato. A posição mais recente é representada por um marcador cuja aparência indica o estado do usuário, e o histórico é exibido como uma trilha, permitindo inferir a direção de deslocamento da vítima. Usuários sem contato por um período superior a alguns intervalos de transmissão são destacados, assim como aqueles que encerraram o alerta.

Como o Sniffer opera justamente em áreas sem cobertura de rede, os mapas devem estar previamente armazenados no dispositivo (mapas *offline*), baixados antes do deslocamento para a área de busca. Além da posição informada pelo GPS do usuário, o aplicativo exibe o RSSI e a SNR do último pacote recebido de cada usuário. Como a intensidade do sinal aumenta à medida que o receptor se aproxima do transmissor, essa informação auxilia a busca nas situações em que a posição informada é imprecisa ou desatualizada.



Figura 9. Interface do aplicativo Sniffer *(em desenvolvimento)*: mapa com a última posição, a trilha e o estado de cada usuário em alerta.

Fonte: elaborado pelos autores.

3.4.3. Comunicação UART no Android

A comunicação entre os aplicativos e o nó é estabelecida quando o dispositivo Android opera em modo USB *host* (USB OTG) e o conversor CH340X da placa enumera-se como um dispositivo serial. Como o sistema Android não expõe nativamente uma abstração de porta serial para aplicações, recorre-se à biblioteca de código aberto *usb-serial-for-android* [mik3y e contribuidores, 2024], que implementa, em espaço de usuário e sobre a API USB *Host* do Android [Google, 2024b], os *drivers* de diversos conversores USB–serial, incluindo a família CH34x utilizada pela placa. Essa abordagem dispensa privilégios de *root* ou alterações no sistema operacional.

O fluxo de inicialização segue as etapas usuais dessa biblioteca: a enumeração dos dispositivos USB conectados, a solicitação de permissão de acesso ao usuário, a abertura do *driver* correspondente ao CH340 e a configuração dos parâmetros da porta (taxa de transmissão, número de bits de dados, paridade e bits de parada), que devem coincidir com os valores definidos no *firmware*. Estabelecida a conexão, a leitura é realizada de forma assíncrona por um leitor dedicado executado em segundo plano, por meio de corrotinas de Kotlin, evitando o bloqueio da interface gráfica, enquanto o envio é feito escrevendo-se diretamente na porta.

Os *bytes* recebidos são acumulados até que se disponha de um quadro íntegro, segundo o protocolo de enquadramento descrito na Subseção 3.3.5, e somente então são interpretados. Essa camada é implementada como um componente compartilhado pelos dois aplicativos, isolando os detalhes do protocolo serial do restante da aplicação, de modo que a diferença entre Beacon e Sniffer se restringe à interface e ao papel enviado ao nó no quadro `CONFIG`.

4. Testes e resultados

5. Conclusão e trabalhos futuros

Referências

Agência Nacional de Telecomunicações (Anatel). Ato nº 14448, de 2017: Requisitos técnicos para a avaliação da conformidade de equipamentos de radiocomunicação de radiação restrita, 2017. Disponível em: https://informacoes.anatel.gov.br/legislacao/. Acesso em: out. 2026.

Agência Nacional de Telecomunicações (Anatel). Anatel explica: Cobertura e zona de sombra, 2021. Edição 91. Disponível em: https://sistemas.anatel.gov.br/anexar-api/publico/anexos/download/810be705911165f93214cf7468570ebc. Acesso em: out. 2026.

Agência Nacional de Telecomunicações (Anatel). Ato nº 14158, de 2025, 2025. Disponível em: https://informacoes.anatel.gov.br/legislacao/atos-de-certificacao-de-produtos/2025/2070-ato-14158. Acesso em: out. 2026.

Agência Nacional de Telecomunicações (Anatel). Ato nº 4746, de 2026, 2026a. Disponível em: https://informacoes.anatel.gov.br/legislacao/atos-de-certificacao-de-produtos/2026/2132-ato-4746. Acesso em: out. 2026.

Agência Nacional de Telecomunicações (Anatel). Plano estrutural de redes de telecomunicações (PERT) – 2026, 2026b. Disponível em: https://www.gov.br/anatel. Acesso em: out. 2026.

A. Augustin, J. Yi, T. Clausen, and W. M. Townsley. A study of LoRa: Long range & low power networks for the Internet of Things. *Sensors*, 16(9):1466, 2016. doi: 10.3390/s16091466.

E. Bäumker, A. M. Garcia, and P. Woias. Minimizing power consumption of LoRa and LoRaWAN for low-power wireless sensor nodes. In *Journal of Physics: Conference Series*, volume 1407, page 012092. IOP Publishing, 2019. doi: 10.1088/1742-6596/1407/1/012092.

Riccardo Berto, Paolo Napoletano, and Marco Savi. A LoRa-based mesh network for peer-to-peer long-range communication. *Sensors*, 21(13):4314, 2021. doi: 10.3390/s21134314.

Brasil. Lei nº 13.709, de 14 de agosto de 2018, 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: out. 2026.

Brasil. Defesa civil alerta, 2024a. Disponível em: https://www.gov.br/mdr/pt-br/assuntos/protecao-e-defesa-civil/defesa-civil-alerta. Acesso em: abr. 2026.

Brasil. Plano nacional de proteção e defesa civil, 2024b. URL https://www.gov.br/mdr/pt-br/assuntos/protecao-e-defesa-civil.

Canaltech. SOS emergência do iPhone ajuda em resgate após avalanche, 2026. Disponível em: https://canaltech.com.br/smartphone/sos-emergencia-do-iphone-ajuda-em-resgate-apos-avalanche/. Acesso em: out. 2026.

Stephen Cass. It's meshtastic! LoRa-based tech brings mesh radio to makers. *IEEE Spectrum*, 61:16–18, 2024. doi: 10.1109/MSPEC.2024.10551793.

Stephanie E. Chang, Timothy L. McDaniels, J. Mikawoz, and K. Peterson. Infrastructure failure interdependencies in extreme events: Power outage consequences in the 1998 ice storm. *Natural Hazards*, 41(2):337–358, 2007. doi: 10.1007/s11069-006-9039-4.

Chengdu Ebyte Electronic Technology Co., Ltd. *MBL Series Evaluation Kit User Manual: E77-900MBL-01*, 2022. URL `https://www.cdebyte.com`. Versão 1.0.

Chengdu Ebyte Electronic Technology Co., Ltd. *E77-900M22S User Manual (STM32WLE5CCU6 Sub-1G SoC Module)*, 2024. URL `https://www.cdebyte.com/products/E77-900M22S`.

Shilpa Devalal and A. Karthikeyan. LoRa technology – an overview. In *Second International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, pages 284–290. IEEE, 2018. doi: 10.1109/ICECA.2018.8474715.

Jefferson Dias and António Grilo. LoRaWAN mesh networks: A review and classification of multihop communication. *Sensors*, 20(15):4273, 2020. doi: 10.3390/s20154273.

M. Dworkin. Recommendation for block cipher modes of operation: The CCM mode for authentication and confidentiality. NIST Special Publication 800-38C, National Institute of Standards and Technology (NIST), 2004.

Zayan El Khaled and Hamid Mcheick. Case studies of communications systems during harsh environments: A review of approaches, weaknesses, and limitations to improve quality of service. *International Journal of Distributed Sensor Networks*, 15(2), 2019. doi: 10.1177/1550147719829960.

Stefano Ferretti et al. Design and performance evaluation of a LoRa-based mobile emergency management system (LOCATE). *Journal of Network and Computer Applications*, 140:82–93, 2019. doi: 10.1016/j.jnca.2019.05.002.

Google. Foreground services overview – android developers, 2024a. Disponível em: `https://developer.android.com/develop/background-work/services/fgs`. Acesso em: out. 2026.

Google. USB host overview – android developers, 2024b. Disponível em: `https://developer.android.com/develop/connectivity/usb/host`. Acesso em: out. 2026.

Jasper Goos et al. Infrastructure failure cascades quintuple risk of storm and flood-induced service disruptions across the globe. *One Earth*, 7(4), 2024. doi: 10.1016/j.oneear.2024.03.012.

George Haddow and Kim S. Haddow. *Disaster Communications in a Changing Media World*. Butterworth-Heinemann, Oxford, 2 edition, 2014. ISBN 978-0-12-407868-0.

Jonas Höchst, Lars Baumgärtner, Franz Kuntke, and Bernd Freisleben. LoRa-based device-to-device smartphone communication for crisis scenarios. In *International Conference on Information Systems for Crisis Response and Management (ISCRAM)*, 2020.

LoRa Alliance. LoRaWAN 1.1 specification, 2017.

LoRa Alliance. RP002-1.0.4 LoRaWAN regional parameters, 2022.

Daniel Lundell, Anna Hedberg, Christian Nyberg, and Emma Fitzgerald. A routing protocol for LoRa mesh networks. In *IEEE 19th International Symposium on A World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, pages 14–19. IEEE, 2018. doi: 10.1109/WoWMoM.2018.8449743.

Khazmir Camille Valerie G. Macaraeg, Calvin Artemies G. Hilario, and Cyrel D. C. Ambatali. LoRa-based mesh network for off-grid emergency communications. In

IEEE Region 10 Conference (TENCON), pages 168–173. IEEE, 2021. doi: 10.1109/TENCON51831.2021.9707429.

MapBiomas. Relatório anual do fogo (RAF): área queimada no Brasil entre 1985 e 2024, 2025. Disponível em: https://brasil.mapbiomas.org. Acesso em: out. 2026.

mik3y e contribuidores. usb-serial-for-android. https://github.com/mik3y/usb-serial-for-android, 2024. Biblioteca de código aberto. Acesso em: jun. 2026.

C. Siva Ram Murthy and B. S. Manoj. *Ad Hoc Wireless Networks: Architectures and Protocols*. Prentice Hall, Upper Saddle River, 2004. ISBN 978-0-13-147023-1.

Nanjing Micro One Electronics Inc. *ME6211 Series: 500 mA Low-Dropout Linear Regulator*, 2017. URL https://www.microne.com.cn.

Nanjing Qinheng Microelectronics Co., Ltd. (WCH). *CH340 Datasheet: USB to Serial Chip*, 2018. URL https://www.wch-ic.com.

NSC Total. Trilheiro que sobreviveu 6 dias na mata reencontra grupo de resgate em Joinville, 2026. Disponível em: https://www.nsctotal.com.br/?p=7933375. Acesso em: out. 2026.

Charles Perkins, Elizabeth Belding-Royer, and Samir Das. Ad hoc on-demand distance vector (AODV) routing. RFC 3561, IETF, 2003.

Charles E. Perkins. *Ad Hoc Networking*. Addison-Wesley Professional, Boston, 2001. ISBN 978-0-201-30976-8.

John C. Pine. *Technology and Emergency Management*. Wiley, Hoboken, 2 edition, 2018. ISBN 978-1-119-43338-7.

Semtech Corporation. SX1276/77/78/79 – LoRa transceiver datasheet, 2019.

STMicroelectronics. *RM0461: STM32WLEx advanced Arm-based 32-bit MCUs with sub-GHz radio solution*. STMicroelectronics, 2024. Reference manual.

Nagender Kumar Suryadevara and Anurag Dutta. Meshtastic infrastructure-less networks for reliable data transmission to augment Internet of Things applications. In *Wireless and Satellite Systems (WiSATS)*, pages 622–640. Springer, 2022. doi: 10.1007/978-3-030-93398-2_55.

Andrew S. Tanenbaum and David J. Wetherall. *Computer Networks*. Prentice Hall, Upper Saddle River, 5 edition, 2011. ISBN 978-0-13-212695-3.

Anthony M. Townsend and Mitchell L. Moss. Telecommunications infrastructure in disasters: Preparing cities for crisis communications. Technical report, New York University, 2005. Disponível em: https://reliefweb.int/attachments/81bb8c74-cafa-3d71-a8b4-540059d1b216/nyu-disastercommunications1-final.pdf. Acesso em: abr. 2026.

D. Whiting, R. Housley, and N. Ferguson. Counter with CBC-MAC (CCM). RFC 3610, Internet Engineering Task Force (IETF), 2003.

Aaron Wong, Sim Lim Goh, Mohammad Kamrul Hasan, and Shaikh Fattah. Multi-hop and mesh for LoRa networks: Recent advancements, issues, and recommended applications. *ACM Computing Surveys*, 56(3):1–43, 2024. doi: 10.1145/3638241.